Dokumentacja Techniczna

Temat	Gra Roguelike inspirowana grą MegaBonk
Autorzy	Kacper Kałuża, Jakub Krysiński
Grupa	2ID13B

---

1. Opis zastosowanych technologii

Technologia	Opis
Język programowania	C#
Silnik gry	Unity 6.3 (LTS)
Biblioteki	UnityEngine, UnityEngine.AI (NavMesh), UnityEngine.UIElements (UI Toolkit), UnityEngine.InputSystem
System UI	UI Toolkit (UXML + USS)
Nawigacja AI	NavMesh (Unity Navigation)
System wejścia	Unity Input System
Kamera	Cinemachine
Kontroler postaci	Starter Assets — Third Person Controller
Kontrola wersji	Unity Version Control 2.11.4

---

2. Opis implementacji

2.1. `PlayerStats.cs`

Centralna klasa przechowująca wszystkie statystyki gracza (zdrowie, doświadczenie, obrażenia, pancerz, lifesteal itp.). Zarządza mechaniką otrzymywania obrażeń, leczenia, zdobywania doświadczenia oraz awansowania na kolejne poziomy. Pełni rolę „single source of truth" dla danych postaci gracza.

Metody:

- `Start()` — Inicjalizuje zdrowie gracza na wartość maksymalną i odświeża interfejs.
- `Update()` — Aktualizuje czas przetrwania i regenerację zdrowia co klatkę.
- `Heal(float amount)` — Przywraca wskazaną liczbę punktów zdrowia z clampowaniem do wartości maksymalnej.
- `TakeDamage(float damage)` — Zmniejsza zdrowie gracza po uwzględnieniu redukcji od pancerza. Wywołuje `Die()` przy osiągnięciu 0 HP.
- `AddExp(float amount)` — Dodaje punkty doświadczenia z uwzględnieniem mnożnika `xpGain`. Sprawdza warunki awansu na następny poziom.
- `LevelUp()` — Podnosi poziom postaci, zwiększa wymagania EXP o 20% i wywołuje ekran wyboru ulepszeń.
- `UpdateUI()` — Metoda delegowana do zewnętrznego systemu `GameUI` (UI Toolkit).
- `AddDamageDealt(float damage)` — Dodaje wartość do licznika łącznych obrażeń zadanych (statystyki końcowe).
- `AddKill()` — Inkrementuje licznik zabitych przeciwników.
- `Die()` — Wyświetla ekran końcowy gry (DeathScreen) lub przeładowuje scenę.

```
public void TakeDamage(float damage)
{
    float mitigatedDamage = Mathf.Max(1f, damage - armor);
    currentHealth -= mitigatedDamage;
    currentHealth = Mathf.Clamp(currentHealth, 0, maxHealth);
    UpdateUI();

    if (currentHealth <= 0)
    {
        Die();
    }
}
```

```
public void AddExp(float amount)
{
    currentExp += (amount * xpGain);
    while (currentExp >= expToNextLevel)
    {
        LevelUp();
    }
    UpdateUI();
}
```

2.2. `PlayerAttack.cs`

Klasa odpowiedzialna za automatyczny system ataków gracza. Wyszukuje najbliższego przeciwnika w zasięgu za pomocą sfery kolizyjnej, tworzy pocisk skierowany w jego stronę, oblicza obrażenia (w tym trafienia krytyczne) i odtwarza dźwięk ataku.

Metody:

- `Start()` — Inicjalizuje referencje do `PlayerStats` i `AudioSource`.
- `Update()` — Odlicza czas między atakami, wywołuje `AttackNearestEnemy()` w odpowiednich interwałach.
- `AttackNearestEnemy()` — Główna logika ataku: detekcja sferyczna, wybór najbliższego celu, obliczenie obrażeń (w tym crit), utworzenie i inicjalizacja pocisku.
- `OnDrawGizmosSelected()` — Wizualizacja zasięgu ataku w edytorze Unity.

```
void AttackNearestEnemy()
{
    int collidersCount = Physics.OverlapSphereNonAlloc(
        transform.position, stats.attackRange, enemyColliders, enemyLayer);
    if (collidersCount == 0) return;

    GameObject nearest = null;
    float minDist = Mathf.Infinity;

    for (int i = 0; i < collidersCount; i++)
    {
        Collider hit = enemyColliders[i];
        if (hit.CompareTag("Enemy"))
        {
            float sqrDist = (transform.position - hit.transform.position).sq
            if (sqrDist < minDist)
            {
                minDist = sqrDist;
                nearest = hit.gameObject;
            }
        }
    }

    if (nearest != null)
    {
        // ... tworzenie pocisku i obliczanie obrażeń krytycznych
        float finalDamage = stats.attackDamage;
        if (Random.Range(0f, 100f) <= stats.critChance)
        {
            finalDamage *= stats.critDamage;
        }
        proj.Initialize(nearest.transform, stats, finalDamage);
    }
}
```

2.3. `Projectile.cs`

Klasa reprezentująca pocisk wystrzeliwany przez gracza. Pocisk podąża w stronę wybranego celu, zadaje obrażenia przy trafieniu i obsługuje mechanikę lifestealu (kradzieży życia).

Metody:

- `Initialize(Transform targetTransform, PlayerStats shooter, float damageAmount)` — Inicjalizuje pocisk przypisując mu cel, obrażenia i referencję do strzelca.
- `Update()` — Przesuwa pocisk co klatkę w stronę celu. Przy kontakcie zadaje obrażenia, dodaje statystyki i leczy gracza (lifesteal).

```
void Update()
{
    if (target == null) { Destroy(gameObject); return; }

    Vector3 direction = (target.position - transform.position).normalized;
    transform.position += direction * speed * Time.deltaTime;

    if (Vector3.Distance(transform.position, target.position) < 0.5f)
    {
        if (target.TryGetComponent(out EnemyHealth health))
        {
            int intDamage = Mathf.RoundToInt(damage);
            health.TakeDamage(intDamage);
            if (shooterStats != null) shooterStats.AddDamageDealt(intDamage);
            if (shooterStats != null && shooterStats.lifesteal > 0f)
            {
                shooterStats.Heal(damage * shooterStats.lifesteal);
            }
        }
        Destroy(gameObject);
    }
}
```

2.4. `PlayerMovement.cs`

Klasa obsługująca poruszanie się postaci gracza z uwzględnieniem orientacji kamery. Wykorzystuje fizykę Rigidbody do przemieszczania, ogranicza pozycję gracza do granic areny i blokuje przepychanie przez przeciwników.

Metody:

- `Start()` — Inicjalizuje komponent Rigidbody.
- `Update()` — Pobiera wejście od gracza (WSAD) i oblicza kierunek ruchu relatywnie do kamery.
- `FixedUpdate()` — Aktualizuje pozycję i rotację gracza przez Rigidbody. Ogranicza pozycję do granic areny. Blokuje przepychanie przez przeciwników.

2.5. `CameraController.cs`

Kontroler kamery w widoku TPP (trzecia osoba). Podąża za wskazanym obiektem, obsługuje obrót myszą i zapobiega przenikaniu kamery przez elementy otoczenia za pomocą Linecast.

Metody:

- `Start()` — Blokuje i ukrywa kursor myszy.
- `LateUpdate()` — Odczytuje wejście myszy, oblicza nową pozycję kamery i wykonuje test kolizji (Linecast) ze środowiskiem.

2.6. `EnemyAI.cs`

Klasa sztucznej inteligencji przeciwników oparta na NavMesh. Zarządza nawigacją w stronę gracza, animacjami ruchu, zadawaniem obrażeń i efektem odrzutu (knockback).

Metody:

- `Start()` — Wyszukuje gracza po tagu `"Player"`, buforuje referencje do komponentów gracza i inicjuje NavMeshAgent.
- `Update()` — Aktualizuje trasę do gracza, zarządza postojem, atakami (z opóźnieniem), efektem knockback oraz animacjami.
- `OnDrawGizmosSelected()` — Rysuje w edytorze sfery wizualizujące `stopDistance` i `attackRange`.

```
// Fragment logiki ataku i odrzutu
if (distance <= attackRange)
{
    if (attackTimer >= attackRate)
    {
        attackTimer = 0f;
        if (playerStats != null)
            playerStats.TakeDamage(attackDamage);
        if (playerCC != null)
        {
            Vector3 pushDirection = (player.position - transform.position).no
            pushDirection.y = 0;
            playerCC.Move(pushDirection * knockbackDistance);
        }
    }
}
```

2.7. `EnemyHealth.cs`

Zarządza punktami życia przeciwnika, obsługuje otrzymywanie obrażeń z efektem wizualnym migania (flash), procedurę śmierci z odtworzeniem dźwięku, upuszczanie kuli doświadczenia i aktualizację paska zdrowia.

Metody:

- `Start()` — Ustawia zdrowie na maksimum, inicjuje pasek zdrowia i pobiera renderery do efektu hit.
- `TakeDamage(int damage)` — Redukuje HP, odświeża pasek, wywołuje efekt migania i sprawdza warunek śmierci.
- `FlashRoutine()` — Korutyna zmieniająca kolor materiału (MaterialPropertyBlock) na czas `flashDuration`.
- `Die()` — Odtwarza dźwięk, informuje spawner, dodaje zabójstwo do statystyk gracza, tworzy kulę doświadczenia i niszczy obiekt.

```
public void TakeDamage(int damage)
{
    currentHealth -= damage;
    currentHealth = Mathf.Max(currentHealth, 0);
    if (healthBar != null) healthBar.UpdateBar(currentHealth, maxHealth);

    if (flashCoroutine != null) StopCoroutine(flashCoroutine);
    flashCoroutine = StartCoroutine(FlashRoutine());

    if (currentHealth <= 0) Die();
}
```

2.8. `EnemyHealthBar.cs`

Zarządza paskiem zdrowia (UI Slider) przypisanym do przeciwnika. Sprawia, by pasek zawsze był skierowany przodem do kamery (efekt billboardingu).

Metody:

- `Start()` — Przypisuje referencję do kamery głównej.
- `LateUpdate()` — Obraca pasek w stronę kamery.
- `UpdateBar(int current, int max)` — Aktualizuje wartość suwaka na podstawie aktualnego i maksymalnego HP.

2.9. `EnemySpawner.cs`

Zarządza procesem pojawiania się przeciwników na arenie. Kontroluje limit przeciwników, interwał respawnu, oraz dynamicznie skaluje trudność gry co minutę (więcej wrogów, wyższe statystyki).

Metody:

- `Update()` — Aktualizuje timery spawnu i skalowania trudności. Wywołuje `SpawnEnemy()` gdy jest miejsce na kolejnego wroga.
- `SpawnEnemy()` — Losuje pozycję na NavMeshu w promieniu od gracza, tworzy wroga i aplikuje mnożniki trudności na jego statystyki.
- `EnemyDied()` — Zmniejsza licznik aktywnych wrogów (wywoływane z EnemyHealth.Die()).

```
void SpawnEnemy()
{
    for (int i = 0; i < 10; i++)
    {
        Vector2 randomCircle = Random.insideUnitCircle.normalized;
        float distance = Random.Range(minDistance, maxDistance);
        Vector3 spawnPos = player.position + new Vector3(
            randomCircle.x * distance, 0f, randomCircle.y * distance);

        if (NavMesh.SamplePosition(spawnPos, out NavMeshHit hit, 2f, NavMesh.
        {
            GameObject enemy = Instantiate(enemyPrefab, hit.position, Quatern
            // Aplikacja modyfikatorów trudności
            EnemyHealth health = enemy.GetComponent<EnemyHealth>();
            if (health != null)
                health.maxHealth = Mathf.RoundToInt(health.maxHealth * curren
            currentEnemies++;
            return;
        }
    }
}
```

2.10. `ExpOrb.cs`

Klasa reprezentująca kulę doświadczenia upuszczaną przez pokonanych przeciwników. Obiekt jest przyciągany w stronę gracza i po zebraniu przyznaje punkty doświadczenia.

Metody:

- `Start()` — Wyszukuje gracza po tagu.
- `Update()` — Sprawdza odległość do gracza (z uwzględnieniem mnożnika `pickupRadius`), przyciąga kulę i przyznaje EXP przy kontakcie.

2.11. `GameUI.cs`

Kontroler HUD (Heads-Up Display) w trakcie rozgrywki oparty o UI Toolkit. Wyświetla pasek zdrowia z dynamiczną kolorystyką, pasek EXP, poziom, timer przetrwania.

Metody:

- `Start()` — Pobiera referencje do PlayerStats i elementów UI Toolkit (etykiety, paski postępu).
- `Update()` — Co klatkę aktualizuje timer i wszystkie wskaźniki interfejsu.
- `UpdateTimer()` — Formatuje czas do `MM:SS` i przypisuje do etykiety.
- `UpdatePlayerUI()` — Synchronizuje UI z danymi z PlayerStats. Zmienia kolor paska zdrowia w zależności od procentu HP (zielony > 50%, żółty > 20%, czerwony poniżej).

2.12. `PauseMenu.cs`

Menedżer pauzy wykorzystujący UI Toolkit. Obsługuje wstrzymywanie/wznawianie gry (klawisz Escape), ukrywanie/pokazywanie kursora oraz wizualne odliczanie 3-sekundowe przed powrotem do rozgrywki.

Metody:

- `OnEnable()` — Wiąże elementy UI i podpina zdarzenia do przycisków.
- `Update()` — Nasłuchuje naciśnięcia klawisza Escape w celu przełączania stanu pauzy.
- `StartResumeCountdown()` — Uruchamia korutynę z animacją odliczania.
- `ResumeCountdownRoutine()` — Korutyna: wyświetla odliczanie 3→2→1 w czasie rzeczywistym, po czym wznawia grę.
- `Pause()` — Zatrzymuje czas, wyświetla menu pauzy i odblokowuje kursor.
- `LoadMainMenu()` — Ładuje scenę menu głównego.

2.13. `DeathScreenManager.cs`

Menedżer ekranu końcowego (śmierci gracza). Wyświetla podsumowanie statystyk gry i umożliwia restart lub powrót do menu głównego. Zbudowany w oparciu o UI Toolkit.

Metody:

- `Start()` — Parsuje drzewo UI, ukrywa overlay, podpina zdarzenia przycisków.
- `OnDestroy()` — Odpina zdarzenia od przycisków (zapobieganie wyciekom pamięci).
- `ShowDeathScreen()` — Wyświetla overlay, wstrzymuje czas i odblokowuje kursor.
- `OnRestartClicked()` — Przeładowuje aktualną scenę.
- `OnExitMenuClicked()` — Ładuje scenę MainMenu.
- `UpdateStatsUI()` — Pobiera dane z PlayerStats i formatuje etykiety (czas, zabójstwa, poziom, obrażenia, statystyki).

2.14. `MainMenuManager.cs`

Menedżer menu głównego zbudowany w UI Toolkit. Obsługuje przyciski Play, Authors i Exit Game.

Metody:

- `Awake()` — Pobiera elementy z dokumentu UI i podpina zdarzenia kliknięcia.
- `OnPlayClicked()` — Ładuje scenę gry (ArenaGry).
- `OnAuthorsClicked()` — Wyświetla informacje o autorach.
- `OnExitClicked()` — Zamyka aplikację.

2.15. `UIUpgradeManager.cs`

Menedżer systemu ulepszeń (Level Up). Po awansie na kolejny poziom wstrzymuje grę i prezentuje trzy losowo wygenerowane karty ulepszeń z różnymi rzadkościami (Common → Legendary). Gracz może wybrać ulepszenie lub je zignorować.

Metody:

- `OnEnable()` — Konfiguruje UI, podpina zdarzenia kliknięcia kart.
- `TriggerLevelUp()` — Zatrzymuje czas, wyświetla overlay i generuje 3 karty ulepszeń.
- `GenerateUpgrades()` — Losuje typ i rzadkość trzech ulepszeń.
- `GenerateRarity()` — Generuje rzadkość na podstawie wagowanych progów (50% Common, 30% Uncommon, 15% Rare, 4% Epic, 1% Legendary).
- `CreateUpgradeData(UpgradeType, UpgradeRarity)` — Tworzy obiekt danych ulepszenia z wartością, opisem i kolorem.
- `SetupCard(Button, UpgradeData)` — Konfiguruje wizualną kartę w UI.
- `ApplyUpgrade(int index)` — Aplikuje wybraną nagrodę na statystyki gracza.
- `IgnoreOffers()` — Zamyka panel bez wyboru ulepszenia.
- `ClosePanel()` — Zamyka overlay, wznawia czas i blokuje kursor.

W pliku zdefiniowane są również typy pomocnicze:

- `UpgradeType` (enum) — 12 typów ulepszeń (AttackDamage, AttackSpeed, MaxHealth, CritChance, itp.).
- `UpgradeRarity` (enum) — 5 poziomów rzadkości (Common, Uncommon, Rare, Epic, Legendary).
- `UpgradeData` (klasa) — Kontener danych ulepszenia (typ, rzadkość, wartość, opis, kolor).

2.16. `ThirdPersonController.cs` (Starter Assets)

Główny kontroler postaci w widoku trzecioosobowym (pakiet Starter Assets). Zarządza ruchem, skokami, grawitacją, animacjami i współpracą z Cinemachine.

Metody:

- `Awake()` — Wyszukuje referencję do kamery głównej.
- `Start()` — Inicjalizuje komponenty (Animator, CharacterController, Input, PlayerStats).
- `Update()` — Główna pętla: skok, sprawdzanie gruntu, ruch.
- `LateUpdate()` — Aktualizacja rotacji kamery.
- `AssignAnimationIDs()` — Hashuje nazwy parametrów animatora dla optymalizacji.
- `GroundedCheck()` — Test sfery sprawdzający kontakt z podłożem.
- `CameraRotation()` — Kontroluje obrót kamery Cinemachine na podstawie wejścia myszy.
- `Move()` — Oblicza i wykonuje ruch postaci z interpolacją prędkości i uwzględnieniem mnożnika z `PlayerStats`.
- `JumpAndGravity()` — Zarządza fizyką skoku i grawitacji z timeoutami.
- `ClampAngle()` — Normalizuje kąty w pętli 360°.
- `OnFootstep(AnimationEvent)` — Odtwarza dźwięk kroku (AnimationEvent).
- `OnLand(AnimationEvent)` — Odtwarza dźwięk lądowania (AnimationEvent).

2.17. `StarterAssetsInputs.cs` (Starter Assets)

Klasa pośrednicząca między Unity Input System a kontrolerem postaci. Zbiera dane wejściowe (ruch, skok, sprint, celowanie) i udostępnia je jako zmienne publiczne.

Metody:

- `OnMove(InputValue)` — Callback wejścia ruchu.
- `OnLook(InputValue)` — Callback wejścia celowania.
- `OnJump(InputValue)` — Callback skoku.
- `OnSprint(InputValue)` — Callback sprintu.
- `MoveInput(Vector2)` — Ustawia wektor ruchu.
- `LookInput(Vector2)` — Ustawia wektor celowania.
- `JumpInput(bool)` — Ustawia stan skoku.
- `SprintInput(bool)` — Ustawia stan sprintu.
- `OnApplicationFocus(bool)` — Zarządza stanem kursora przy zmianie fokusu okna.
- `SetCursorState(bool)` — Blokuje/odblokowuje kursor.

2.18. `BasicRigidBodyPush.cs` (Starter Assets)

Skrypt umożliwiający przepychanie obiektów z komponentem Rigidbody przez gracza wyposażonego w CharacterController.

Metody:

- `OnControllerColliderHit(ControllerColliderHit)` — Callback kolizji kontrolera z otoczeniem.
- `PushRigidBodies(ControllerColliderHit)` — Oblicza kierunek i aplikuje impuls fizyczny na trafiony obiekt (tylko w poziomie, tylko na dozwolonych warstwach).

2.19. `PlayerInput.cs`

Klasa przejściowa pobierająca akcje wejścia z nowego systemu Input System Unity (Move, Jump, Look).

Metody:

- `Start()` — Podłącza akcje wejścia (Move, Jump, Look) z globalnego InputSystem.
- `Update()` — Sprawdza stan skoku i deleguje czytanie ruchu.
- `FixedUpdate()` — Odczytuje wartość ruchu z bufora systemu wejściowego.

---

3. Instrukcja dla dewelopera

3.1. Narzędzia i środowisko

Narzędzie	Platforma
Silnik	Unity 6.3 (LTS)
Edytor kodu (macOS)	Visual Studio Code
Edytor kodu (Windows)	Microsoft Visual Studio
Kontrola wersji	Unity Version Control 2.11.4

3.2. Wymagane pakiety Unity

- **Starter Assets** — do zarządzania kontrolerem postaci gracza (PlayerRoot).
- **Cinemachine** — do obsługi systemu kamer (śledzenie, kolizje).
- **Input System** — do obsługi wejścia od gracza (klawiatura, mysz, gamepad).
- **UI Toolkit** — do tworzenia interfejsów użytkownika (HUD, menu, ekrany).

3.3. Struktura folderów projektu

```
Assets/
├── Enemy/              – Zasoby przeciwników (modele, materiały, prefaby)
├── Map/                – Zasoby mapy (tereny, elementy otoczenia)
├── Music/              – Muzyka w tle
├── Player/             – Zasoby gracza
│   ├── PlayerInput.cs  – Skrypt wejścia gracza
│   └── StarterAssetsCore/
│       ├── InputSystem/
│       │   └── StarterAssetsInputs.cs
│       └── Scripts/
│           ├── BasicRigidBodyPush.cs
│           └── ThirdPersonController.cs
├── Prefabs/            – Prefabrykaty obiektów gry
├── Scenes/             – Sceny Unity
│   ├── ArenaGry.unity  – Główna scena rozgrywki
│   ├── MainMenu.unity  – Scena menu głównego
│   └── MainMenuManager.cs
├── Scripts/            – Główne skrypty gry
│   ├── CameraController.cs
│   ├── DeathScreenManager.cs
│   ├── EnemyAI.cs
│   ├── EnemyHealth.cs
│   ├── EnemyHealthBar.cs
│   ├── EnemySpawner.cs
│   ├── ExpOrb.cs
│   ├── GameUI.cs
│   ├── PauseMenu.cs
│   ├── PlayerAttack.cs
│   ├── PlayerMovement.cs
│   ├── PlayerStats.cs
│   └── Projectile.cs
├── Settings/           – Ustawienia projektu
├── Sounds/             – Efekty dźwiękowe
├── UI/                 – Zasoby UI Toolkit
│   ├── UIUpgradeManager.cs
│   ├── DeathScreen.uxml / .uss
│   ├── GameHUD.uxml / GameHUDStyles.uss
│   ├── MainMenu.uxml / .uss
│   ├── PauseMenu.uxml / .uss
│   ├── UpgradeScreen.uxml / UpgradeStyles.uss
│   └── MainPanelSettings.asset
├── UI Toolkit/         – Domyślne zasoby UI Toolkit
├── arena_gry/          – Dodatkowe zasoby areny
└── EXP.prefab          – Prefabrykat kuli doświadczenia
```

3.4. Budowa interfejsu UI (UI Toolkit)

Wszystkie ekrany interfejsu użytkownika zostały zbudowane przy użyciu **UI Toolkit**. Każdy widok składa się z pary plików:

- **`.uxml`** — Struktura dokumentu (odpowiednik HTML).
- **`.uss`** — Style wizualne (odpowiednik CSS).

Elementy UI są wyszukiwane w kodzie C# za pomocą metody `root.Q<T>("NazwaElementu")` i obsługiwane poprzez zdarzenia (np. `button.clicked += callback`).

3.5. Kompilacja i uruchomienie

1. Otwórz projekt w **Unity 6.3** (LTS).
2. Zainstaluj wymagane pakiety z Package Manager (jeśli brakujące):
   - Input System
   - Cinemachine
   - Starter Assets
3. Otwórz scenę `Assets/Scenes/ArenaGry.unity` (główna rozgrywka) lub `Assets/Scenes/MainMenu.unity` (menu startowe).
4. Kliknij przycisk **Play** w edytorze Unity, aby uruchomić grę.
5. Do zbudowania aplikacji wykonywalnej: **File → Build Settings → Build**.

---

4. Instrukcja obsługi

4.1. Uruchomienie gry

Po uruchomieniu gry wyświetla się **Menu Główne** z trzema przyciskami:

- **Play** — Rozpoczyna nową grę.
- **Authors** — Wyświetla informacje o autorach (Kacper Kałuża i Jakub Krysiński).
- **Exit Game** — Zamyka aplikację.

4.2. Sterowanie

Klawisz	Akcja
W / A / S / D	Poruszanie się postacią
Mysz	Obrót kamery
Shift	Sprint
Spacja	Skok
Escape	Menu pauzy

Ataki wykonywane są **automatycznie** — gracz strzela pociskami w najbliższego przeciwnika w zasięgu.

4.3. Interfejs gry (HUD)

Podczas rozgrywki na ekranie wyświetlane są następujące elementy:

- **Pasek zdrowia** (lewy dolny róg) — wizualna reprezentacja aktualnych punktów zdrowia. Kolor zmienia się dynamicznie:
  - **Zielony** — powyżej 50% HP
  - **Żółty** — od 20% do 50% HP
  - **Czerwony** — poniżej 20% HP
- **Liczba zdrowia** (nad paskiem zdrowia) — wyświetla wartość w formacie `aktualne / maksymalne` (np. `80 / 100`).
- **Timer** (górna część ekranu) — odmierza czas przetrwania od początku rozgrywki w formacie `MM:SS`.
- **Pasek doświadczenia (EXP)** (prawy dolny róg) — wizualizacja postępu do kolejnego poziomu.
- **Aktualny / wymagany EXP** (nad paskiem EXP) — wyświetla liczbę zebranych punktów do kolejnego poziomu (np. `0 / 12`).
- **Poziom postaci** (prawy dolny róg, nad paskiem EXP) — wyświetla aktualny poziom gracza (np. `LVL 2`).

Każdy pokonany przeciwnik posiada nad sobą **czerwony pasek zdrowia** wskazujący jego pozostałe HP.

4.4. System ulepszeń (Level Up)

Po zebraniu wystarczającej ilości doświadczenia gracz awansuje na kolejny poziom. Gra zostaje wstrzymana i wyświetlane są **trzy karty ulepszeń** o losowej rzadkości:

- **Common** (szary) — podstawowe wzmocnienie
- **Uncommon** (zielony) — umiarkowane wzmocnienie
- **Rare** (niebieski) — znaczące wzmocnienie
- **Epic** (fioletowy) — mocne wzmocnienie
- **Legendary** (złoty) — najsilniejsze wzmocnienie

Gracz może wybrać jedną kartę lub kliknąć **"Ignore Offers"**, aby odrzucić wszystkie i wrócić do gry.

Dostępne ulepszenia obejmują m.in.: obrażenia ataku, szybkość ataku, maksymalne HP, szansę na krytyka, lifesteal, pancerz, prędkość ruchu, regenerację zdrowia, zasięg ataku, bonus XP i zasięg podnoszenia przedmiotów.

4.5. Menu pauzy

Naciśnięcie **Escape** otwiera menu pauzy z dwoma opcjami:

- **Resume** — Wznawianie gry z 3-sekundowym odliczaniem.
- **Exit to Main Menu** — Powrót do ekranu głównego.

4.6. Ekran końcowy (Śmierć)

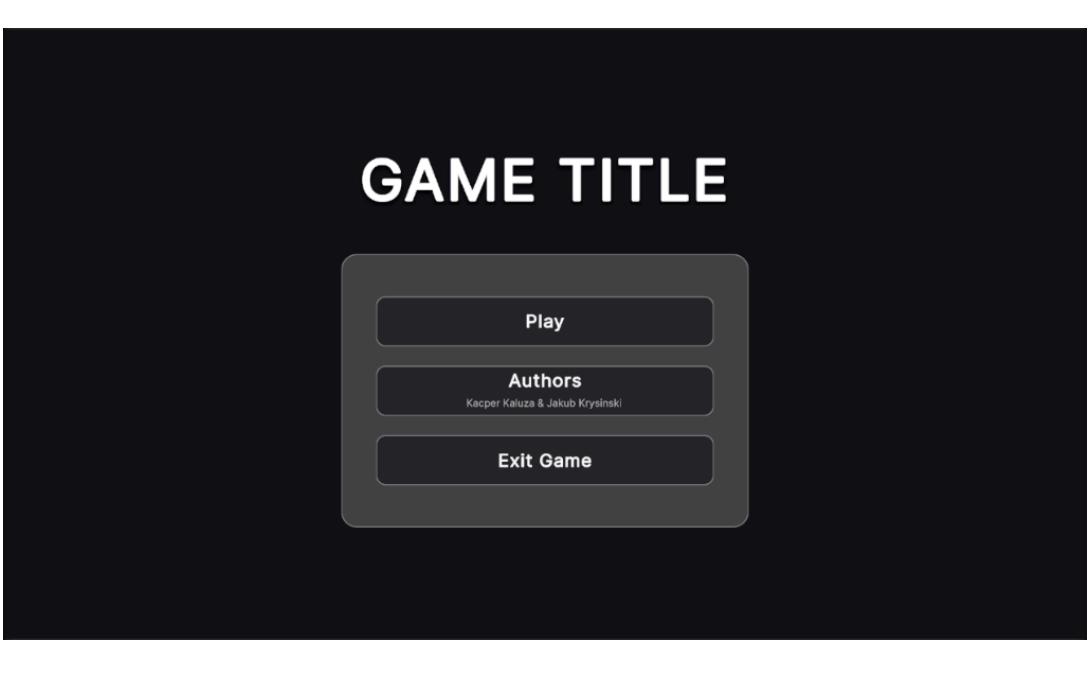
Po utracie wszystkich punktów zdrowia wyświetla się ekran końcowy z podsumowaniem:

- **Game Summary** — czas przetrwania, liczba zabitych wrogów, osiągnięty poziom, łączne zadane obrażenia.
- **Player Stats** — końcowe wartości statystyk gracza (obrażenia, szybkość ataku, HP, pancerz, crit chance, crit dmg, prędkość ruchu).
- Przyciski: **Restart** (ponowna gra) i **Exit to Menu** (powrót do menu głównego).

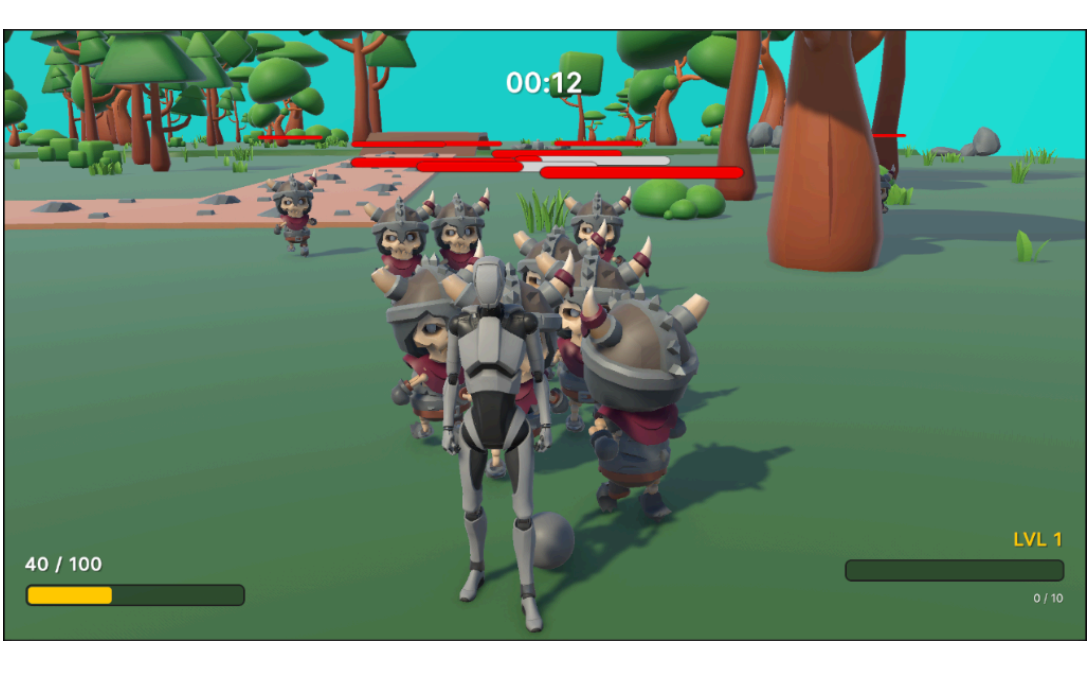
---

5. Przykłady działania — zrzuty ekranu

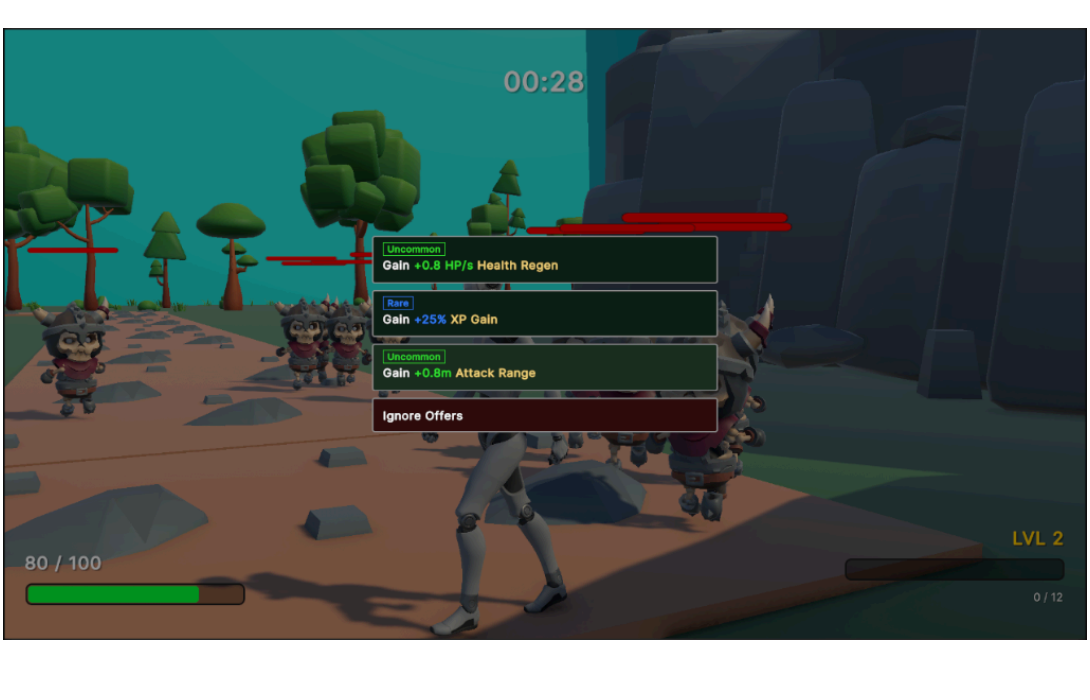
Menu główne



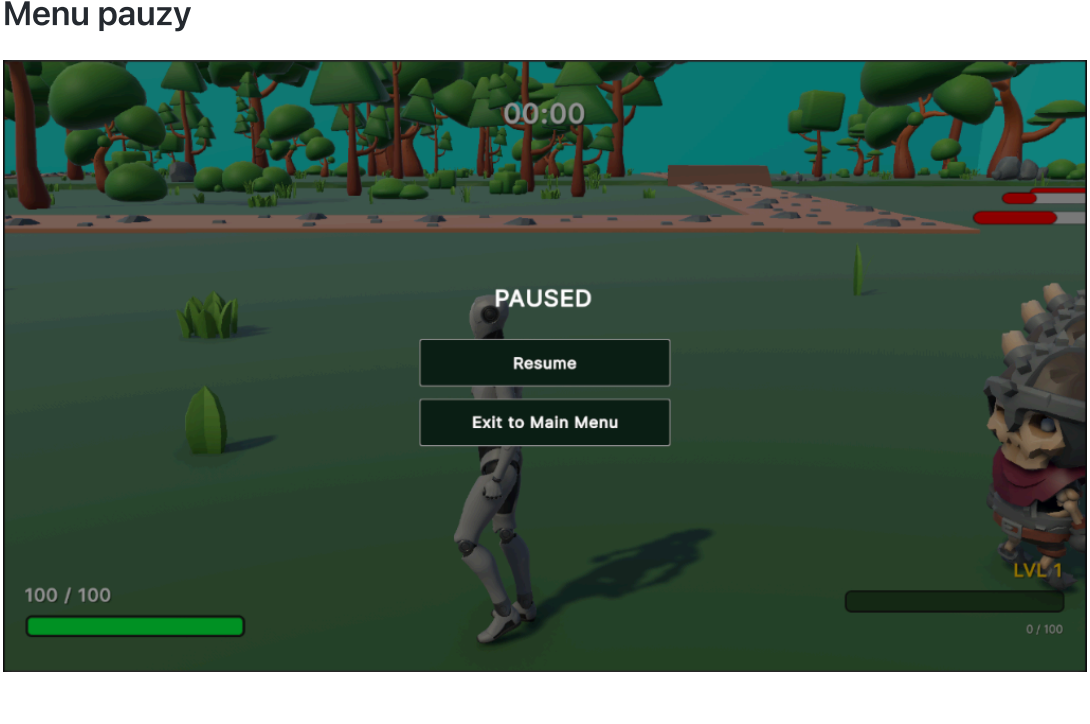
Rozgrywka — walka z przeciwnikami



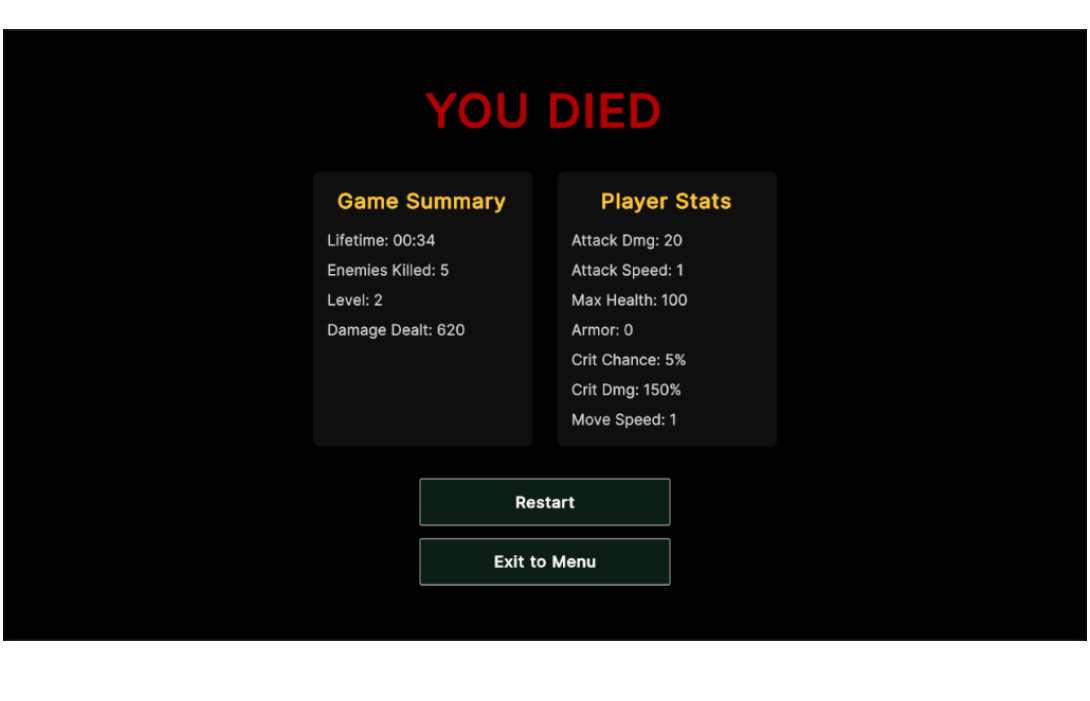
Ekran wyboru ulepszeń (Level Up)



Menu pauzy



Ekran końcowy (Śmierć)



Game Summary
Lifetime: 00:34
Enemies Killed: 5
Level: 2
Damage Dealt: 620

Player Stats
Attack Dmg: 20
Attack Speed: 1
Max Health: 100
Armor: 0
Crit Chance: 5%
Crit Dmg: 150%
Move Speed: 1